StillCold — Final Presentation

Environmental monitoring without opening the door

Jeffrey Perdue · CSC 494 · Spring 2026

What is StillCold?

The problem: Every time you open a refrigerated enclosure to check the temperature, you change what you are measuring — and hide what is actually happening over time.

The solution: An HTU21D sensor wired to a Seeed XIAO ESP32-C6 exposes live temperature and humidity as BLE characteristics. A Flutter companion app polls those characteristics, persists readings locally, and provides a dashboard with alerts and trend charts.

Final system architecture

```
HTU21D ──I²C──▶ ESP32-C6 ──BLE──▶ Flutter App ──▶ User
```

Layer	Component
Sensor	HTU21D temperature + humidity (I^2C, 3.3 V)
MCU / BLE server	Seeed XIAO ESP32-C6
Transport	BLE GATT (custom service + characteristics)
App + storage	Flutter · Drift (SQLite) · fl_chart

Single-MCU design finalised in Sprint 2 — Arduino Nano and level shifter removed.

Live Demonstration

What you will see:

- StillCold advertising as **STILLCOLD** over BLE
- Flutter app discovering, connecting, and displaying live temperature and humidity
- Settings
- Trend charts

Demo Video

Link to Demo Video

Progress since S1P — Overview (Weeks 10–14)

Week	Theme	Outcome
10	Stabilise & Plan	Architecture docs, SRS gap analysis, Sprint 2 backlog
11	Cold-Environment Testing	3 scenarios validated; BLE stable through fridge door
12	Flutter Must + Should	7 features shipped + DB migrations; all core SRS done
13	Single-MCU Migration	Nano + level shifter removed; unified ESP32-C6 firmware
14	Trend Charts	1h/24h/7d charts, downsampling, retention; all SRS tiers complete

Week 10 — Stabilise and Plan

- Wrote the complete hardware architecture document (UART format, BLE GATT table, pin assignments)
- Performed SRS gap analysis: connection lifecycle hard-coded; polling timer not wired; trend chart placeholder only
- Produced the Sprint 2 backlog (Must → Should → Stretch) with a week-by-week execution plan

> *Planning first meant every later week had a clear, scoped target.*

Week 11 — Cold-Environment Testing

Goal: Validate hardware in a real refrigerated environment before touching the app.

Scenario	What was tested	Result
Steady-state (~1 h)	Stable readings inside fridge	**37.7–38.2 °F**; BLE stable throughout
Door events (35 s / 3 min)	Recovery time after door open	+0.9 °F / +2.6 °F; recovered in 12–23 min
Power cycles (×10)	Boot-to-valid-reading time	**Avg 10.8 s** (target < 15 s) ✓

Key finding: stale-data UX must flag readings more than ~2–3 minutes old — shaped all Week 12 design decisions.

Week 12 — Flutter: Must + Should Complete

7 spec-driven features shipped; no hardware changes.

Feature	What it does
Auto-connect	Reconnects to last known device on launch
Onboarding gate	First-run only; returning users skip to dashboard
RSSI display	Live signal strength shown on dashboard
Device labels	Long-press to rename a paired device
24h min/max	Shown at a glance on the dashboard
Stale indicator	Amber "Stale · last updated X min ago" when readings age
Threshold unit fix	°C / °F consistency in threshold dialog + rationale copy

Also fixed: DB migration crash after deploy (v1→v2→v3); 24h min/max SQL query rewrite.

Week 13 — Single-MCU Migration

Removed: Arduino Nano, level shifter, UART wire, and entire text protocol.

Before

```
HTU21D → Nano (5 V) → level shifter → ESP32-C6 → BLE
```

After

```
HTU21D → ESP32-C6 (3.3 V, I²C direct) → BLE
```

Key firmware changes:

- `Wire.setPins(22, 23)` — corrected I^2C pins (D4/D5 on XIAO footprint)
- Error codes **998 / 999** guarded in addition to `isnan()`
- BLE service UUID and value format unchanged

Flutter app required zero changes — the BLE contract was stable throughout.

Week 14 — Trend Charts & All SRS Complete

Stretch FR-3.6 implemented: 1h / 24h / 7d trend charts with temperature / humidity toggle.

- 30-day retention policy; schema **v4** composite index `(deviceId, timestamp)`
- Gap visualisation via `nan` breaks in fl_chart
- Y-axis "nice number" algorithm; outlier clamp; landscape overflow fixed
- `deviceId` scoping bug on min/max fixed

All Must + Should + Stretch SRS requirements complete.

Links

Resource	URL
Canvas	https://nku.instructure.com/courses/88152/pages/individual-project-jeffrey-perdue
GitHub	https://github.com/jeffreyperdue/stillcold-csc-494-iot-project
Learning with AI	https://github.com/jeffreyperdue/csc-494-learning-with-ai

All weekly presentations, firmware, Flutter source, architecture docs, and cold-test reports are in the GitHub repository.

What I Learned with AI — Topic 1

Hardware: Sensors and Data Acquisition

Core insight: A sensor output is an *interpretation*, not an inherent truth.

- The HTU21D samples the physical environment, runs internal ADC circuitry, applies compensation algorithms from its datasheet, then hands a formatted value to the I^2C bus — every step shapes what the number means
- Cold-environment testing made this tangible: bubble wrap insulation slowed sensor cooldown, so the reading was accurate for the *sensor's immediate environment*, not the open fridge air
- AI helped me read datasheets and connect classroom theory (ADC, sampling) to observed behavior on the bench

> *Testing is the only way to know whether the interpretation is actually useful.*

What I Learned with AI — Topic 2

Software: Applications as System Interfaces

Core insight: The Flutter app is not a standalone app — it is a *layer in a physical system*.

- BLE GATT uses a *read* model (poll), not a *push* model; understanding this drove the polling timer, stale-data UX, and connection lifecycle design choices
- AI helped me reason about polling vs notifications and why disconnect handling matters differently when the "server" is a microcontroller that can lose power at any time
- The payoff came in Week 13: because the app never knew the Nano existed and the BLE contract was stable, the entire single-MCU migration required **zero app changes**

> *Good interface design made a hardware refactor invisible to the software.*

Issues Encountered and How I Solved Them

Issue	Week	Resolution
Wrong I^2C pins in migration plan	13	Verified with XIAO pinout before wiring; `Wire.setPins(22, 23)`
DB migration crash after deploy	12	Added explicit migration path v1→v2→v3; settings preserved
24h min/max query returning empty	12	Rewrote SQL; root cause was timestamp range condition
Sensor returning error codes 998/999 not caught by guard	13	Extended guard beyond `isnan()` to handle HTU21D-specific codes
min/max showing cross-device values	14	Added `WHERE deviceId = ?` predicate to query

NKU Celebration of Student Research and Creativity

Thursday, April 23, 2026

StillCold will be presented as a live demonstration at the **NKU Celebration of Student Research and Creativity**.

StillCold Research Poster

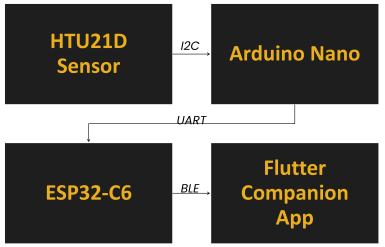
StillCold: Environmental Monitoring Without Opening the Door

Jeffrey Perdue, Dr. Samuel Cho
School of Computing and Analytics, College of Informatics, Northern Kentucky University

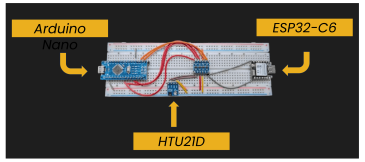
Introduction / Motivation

- Opening a refrigerator to check the temperature disrupts the environment and speeds spoilage; StillCold enables monitoring without opening the door.
- Bacteria can grow at slightly unsafe temperatures before food looks spoiled; StillCold provides continuous, early detection.
- Uses Bluetooth Low Energy (BLE) only—no Wi-Fi or cloud—keeping data local and running on a USB power bank even during outages.

System Architecture



Hardware

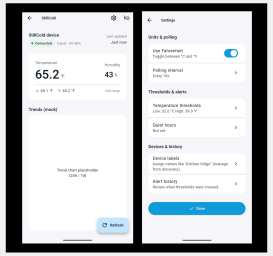


Component	Role
HTU21D Sensor	Measures temp & humidity inside the closed environment
Arduino Nano	Reads the sensor and passes the data along
ESP32-C6	Wirelessly broadcasts the latest temp & humidity to a smartphone via Bluetooth
Anker Power Bank	Powers the entire device

Flutter Companion App

- Runs on Android; discovers and connects to StillCold automatically over Bluetooth Low Energy
- Live dashboard displays current temperature and humidity — no manual refresh needed
- Readings timestamped and stored locally in SQLite (database) — no internet or cloud account required
- Customizable high/low temperature thresholds trigger alerts when conditions drift outside a safe range
- Full alert history log records when thresholds were crossed and the reading at that time

Flutter is a framework made by Google that allows you to write code once, and deploy it across multiple platforms (Android, iOS, Web)



Flutter companion app — live dashboard (left) and alert configuration (right)

How I Used AI to Solve the Problem

StillCold required learning several unfamiliar technologies simultaneously. AI served as an on-demand tutor and thought partner throughout the build.

Learning the Basics

- To learn C++ (the language powering the microcontrollers) and understand how a breadboard works — the platform used to wire and test the hardware prototype.

Building the Hardware

- To reason through wiring decisions, diagnose voltage mismatches, and evaluate component choices before testing them on the bench.

Building the App

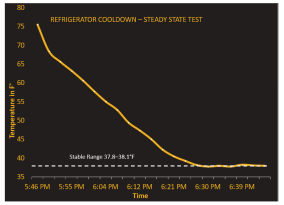
- To accelerate Flutter development, structure the codebase, and translate project requirements into working software features.

The Bigger Picture

- AI was most valuable for asking "why" before "how" — understanding tradeoffs rather than just getting answers. Every decision was ultimately verified through building and testing.

Cold Environment Testing

- Hardware sealed in a Ziploc bag with bubble wrap and placed in a household refrigerator.
- Three scenarios tested: steady-state cooldown, door open/close events, and post-power-cycle reconnection. **All three passed.**



Temperature recorded every 1–2 min via the Flutter app. Dashed line indicates stable fridge range (37.8–38.1°F).

Scenario	Key Result
Short door open (~35 sec)	+0.9°F peak, recovered in ~12 min
Prolonged Door Open	+2.6°F peak, recovered in ~23 min
Power Cycle (10 trials)	Valid reading in 10.8s avg (target: <15s)

Readings remained within ±0.25°F over 16 minutes at steady state — sufficient precision for food safety monitoring.

Next Steps

- Migrate hardware from breadboard to perfboard for a more durable assembly
- Explore single-MCU design by connecting the sensor directly to the ESP32-C6, removing the Arduino Nano
- Temperature history trend charts in the Flutter app
- iOS compatibility verification

Acknowledgements

The author thanks Dr. Samuel Cho for guidance and mentorship throughout this project. This work was supported by the School of Computing and Analytics, College of Informatics, Northern Kentucky University, and CINSAM.

Thank you

StillCold — *Environmental monitoring without opening the door*

- **Canvas:** https://nku.instructure.com/courses/88152/pages/individual-project-jeffrey-perdue
- **GitHub:** https://github.com/jeffreyperdue/stillcold-csc-494-iot-project

Questions?